ComicCraft – Workflow

Project Workflow and End-to-End Process

Project: ComicCraft – AI Comic Story Creator
Technology: FastAPI, Jinja2, Gemini models, Stable Diffusion, FPDF
Topic covered: Workflow of the ComicCraft AI comic-generation application

This assignment is organized as a standalone technical note with clear concepts, implementation points, workflow diagrams, and practical checkpoints.

Prepared for Academic Submission

1. Introduction to the ComicCraft Workflow

ComicCraft is a web-based AI application designed to convert a user's creative idea into a complete comic. The workflow begins with a story prompt and additional preferences such as character name, setting, tone, and art style. The backend then coordinates AI text generation, image generation, layout construction, and PDF export.

Major Workflow Stages

User Input	AI Outline	Story + Dialogue	Images	Layout	PDF Export

- **Input collection:** The frontend collects the story prompt and creative preferences.
- **Structured planning:** Gemini is used to create a panel-by-panel outline.
- **Story expansion:** The outline is expanded into narration and character dialogue.
- **Visual generation:** Stable Diffusion generates illustrations from image prompts.
- **Layout construction:** Images and text are matched panel by panel.
- **Export:** FPDF creates a downloadable multi-page comic PDF.

Workflow Objectives

- Maintain a clear sequence between text generation and image generation.
- Keep every generated panel connected to the same story outline.
- Allow users to review the comic before exporting it.
- Separate frontend, backend, AI integration, and export responsibilities.
- Make the workflow repeatable so a new prompt can generate a new comic.

2. Project Milestones and Activities

Milestone	Main Activities	Purpose
1. Model Selection & Architecture	Research models; define architecture; set up development environment	Choose suitable AI services and establish the technical fou
2. Core Functionalities	Outline, story, image, layout and PDF functions; FastAPI backend	Build the main comic-generation capabilities.
3. routes.py Development	Create routes and handlers; connect workflow functions; error handling	Coordinate requests and backend processing.
4. Frontend Development	HTML/CSS UI; Jinja2 templates; preview and export pages	Provide an interactive user experience.
5. Deployment	Local configuration; environment variables; testing and verification	Run the complete application locally before wider deploym

End-to-End Request Flow

Browser	/generate	generate_outline()	generate_story()	generate_image()	save_pdf()

The main /generate route acts as the central orchestration point. It receives the form data, calls the generation functions in sequence, builds the final layout, and sends the result to the preview template. A JSON route provides a similar pipeline for API clients.

Important Workflow Principle

Each stage should produce a well-defined output that can be consumed by the next stage. For example, the outline supplies panel descriptions and image prompts; the story function uses the outline to produce narration; the image generator uses image prompts; and the layout builder combines the resulting assets.

3. Frontend, Backend and AI Interaction

ComicCraft follows a layered architecture. The frontend is responsible for collecting input and displaying results. FastAPI provides the backend and routing layer. AI services perform story and image generation, while utility modules prepare the final comic for export.

Layer	Components	Responsibilities
Frontend	HTML, CSS, Jinja2	Collect user details, submit requests, display panels and export status.
Backend	FastAPI, routes.py	Validate input, call functions, manage routes and handle errors.
Text AI	Gemini models	Create structured outlines, narration and dialogue.
Image AI	Stable Diffusion / Diffusers	Generate comic-style illustrations from prompts.
Assembly	layout_builder.py	Match images and text into a structured panel list.
Export	exporters.py / FPDF	Create the final multi-page PDF.

Example User Journey

- The user enters: “A brave fox exploring an enchanted forest.”
- The user chooses a character name, setting, tone and art style.
- The application generates a structured five-panel outline.
- The outline becomes narration and dialogue.
- Each panel receives an image prompt and generated illustration.
- The application assembles the panels into a preview.
- The user exports the finished comic as a PDF.

Extra Point: Iteration

A useful feature of the workflow is iteration. If the user changes the tone from dramatic to funny or changes the art style, the generation pipeline can be run again using the updated input. This makes the application customizable instead of producing only one fixed result.

4. Workflow Quality, Validation and Error Handling

A complete workflow should not only generate output; it should also verify that each stage has produced usable data. Validation reduces failures caused by missing inputs, invalid API responses, or unavailable generated files.

Suggested Validation Checkpoints

Stage	Validation Check	Expected Result
Input	Required fields are present	Valid story prompt and preferences.
Outline	Panel count and required fields checked	Each panel has number, title, description and image prompt.
Story	Narration is available for every panel	Readable story text and dialogue.
Image	Generated file exists	Each panel has an image path.
Layout	Text and image indexes match	Correct image paired with correct panel.
Export	PDF file created successfully	Downloadable final document.

Error Handling

- Use try-except blocks around external AI calls and file operations.
- Use FastAPI HTTPException for controlled API errors.
- Return useful error messages instead of exposing internal implementation details.
- Check that generated image files exist before attempting PDF export.
- Keep temporary/generated files organized in dedicated folders.

Performance and Maintainability

- Avoid mixing UI code with AI-generation logic.
- Keep model-specific functions in separate modules.
- Use reusable helper functions for layout and export.
- Log important events such as request start, model call, image creation and export completion.

5. Summary and Workflow Checklist

The ComicCraft workflow connects creative user input with a sequence of AI and software components. Its main strength is the separation of responsibilities: the frontend collects information, FastAPI coordinates processing, Gemini generates text, Stable Diffusion generates images, and FPDF creates the final document.

Workflow Checklist

- ■ User input form is available and clearly labeled.
- ■ FastAPI receives and validates the input.
- ■ Structured comic outline is generated.
- ■ Narration and dialogue are produced.
- ■ Image prompts are converted into illustrations.
- ■ Images and text are assembled in the correct order.
- ■ Preview page displays the generated comic.
- ■ PDF export creates a readable final document.
- ■ Errors are handled gracefully.
- ■ Local testing confirms the full workflow.

Conclusion

A well-designed workflow turns ComicCraft from a collection of independent AI functions into a complete application. The sequential pipeline also makes development easier because each milestone can be tested independently before the entire system is connected.

Reference: Project material supplied in the ComicCraft document; local-development practices were also cross-checked against GitHub and Google Cloud developer documentation.